CENTROID-BASED TEXT CLASSIFICATION

Python NLP Laboratory — Step-by-step execution and explanation

Objective: Calculate a centroid for each category using training documents and classify new documents based on their similarity to the category centroids.

1. What is Centroid-Based Text Classification?

Centroid-Based Classification is a Machine Learning technique that classifies documents based on their similarity to the centre of each category. A centroid represents the average numerical feature vector of all documents belonging to a particular category.

Example categories are Technology, Sports and Education. The computer learns from labelled training documents, calculates one centroid for each category and compares a new document with these centroids to select the most similar category.

2. Understanding a centroid

A centroid is the mathematical centre of a group of data points. Suppose three documents have two numerical features and are represented as D1 = (2, 1), D2 = (4, 2), and D3 = (6, 3).

Cx = (2 + 4 + 6) / 3 = 4
Cy = (1 + 2 + 3) / 3 = 2
Therefore, the centroid is C = (4, 2).

In text classification, we use many word-based features generated through TF-IDF instead of only two numerical features.

3. How does centroid-based text classification work?

Workflow: labelled training documents → TF-IDF feature extraction → calculate category centroids → calculate similarity with each centroid → predict the category with the highest similarity.

The training documents have predefined category labels. The centroid is calculated by averaging the TF-IDF vectors of the documents in each category.

4. What is TF-IDF?

TF-IDF stands for Term Frequency–Inverse Document Frequency. It converts text documents into numerical feature vectors and assigns weights to words based on their frequency in a document and across the document collection. These vectors are used to calculate category centroids.

5. How is the centroid calculated?

Suppose a category contains three training documents represented by these vectors:

D1 = [0.2, 0.4, 0.1]
D2 = [0.4, 0.2, 0.3]
D3 = [0.3, 0.3, 0.2]

C = (D1 + D2 + D3) / 3
C = [(0.2+0.4+0.3)/3, (0.4+0.2+0.3)/3, (0.1+0.3+0.2)/3]
Centroid = [0.3, 0.3, 0.2]

The same averaging procedure is performed for every category.

6. How do we classify a new document?

After calculating the centroids, compare a new document with each category centroid using cosine similarity:

Cosine(A, B) = (A · B) / (||A|| × ||B||)

A is the TF-IDF vector of the new document; B is a category centroid; A · B is the dot product; and ||A|| and ||B|| are vector magnitudes. The category with the highest cosine similarity is selected.

Illustrative scores: Technology = 0.85, Sports = 0.22, Education = 0.15. The predicted category is Technology because it has the highest similarity.

7. Install the required Python libraries

Open Command Prompt or the VS Code terminal and execute:

```
pip install scikit-learn numpy
```

The program uses TfidfVectorizer, train_test_split, cosine_similarity, NumPy and accuracy_score/classification_report.

8. Complete Python program

Create a Python file named **centroid_text_classification.py** and enter the following code:

```
# CENTROID-BASED TEXT CLASSIFICATION

# STEP 1: Import required libraries
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report

# STEP 2: Create sample training documents
documents = [
    # TECHNOLOGY DOCUMENTS
    "Artificial intelligence is transforming healthcare",
    "Machine learning is used in data science",
    "Python programming is useful for software development",
    "Deep learning is a branch of artificial intelligence",
    "Computer networks connect different devices",

    # SPORTS DOCUMENTS
    "India won the cricket match",
    "Football is a popular sport",
    "The cricket team scored a century",
    "The football player scored a goal",
    "The team won the basketball tournament",

    # EDUCATION DOCUMENTS
    "Students attend classes in college",
    "Teachers explain programming concepts",
    "University conducts semester examinations",
    "Students prepare assignments and projects",
    "Online education provides learning opportunities"
]

# STEP 3: Define category labels
labels = [
    "Technology", "Technology", "Technology",
    "Technology", "Technology",
    "Sports", "Sports", "Sports",
    "Sports", "Sports",
    "Education", "Education", "Education",
    "Education", "Education"
]

# STEP 4: Divide dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    documents, labels,
    test_size=0.2,
    random_state=42,
    stratify=labels
)

# STEP 5: Convert text into TF-IDF features
vectorizer = TfidfVectorizer(stop_words="english")
X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)

# STEP 6: Identify unique categories
categories = np.unique(y_train)

# STEP 7: Calculate centroid for each category
```

```
centroids = []

for category in categories:
    category_documents = X_train_tfidf[
        np.array(y_train) == category
    ]
    centroid = category_documents.mean(axis=0)
    centroids.append(np.asarray(centroid).ravel())

# Convert centroids into a matrix
centroids = np.array(centroids)

print("\n===== CATEGORY CENTROIDS =====")
for category, centroid in zip(categories, centroids):
    print("\nCategory:", category)
    print("Centroid:", centroid)

# STEP 8: Calculate similarity with category centroids
similarity_matrix = cosine_similarity(
    X_test_tfidf, centroids
)

# STEP 9: Predict category based on highest similarity
predicted_indices = np.argmax(similarity_matrix, axis=1)
y_pred = categories[predicted_indices]

# STEP 10: Display test document predictions
print("\n===== TEST DOCUMENT CLASSIFICATION =====")
for document, actual, predicted in zip(X_test, y_test, y_pred):
    print("\nDocument:", document)
    print("Actual Category:", actual)
    print("Predicted Category:", predicted)

# STEP 11: Evaluate classification accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\n===== CLASSIFICATION PERFORMANCE =====")
print("Accuracy:", round(accuracy, 4))
print("\nClassification Report:")
print(classification_report(y_test, y_pred, zero_division=0))

# STEP 12: Classify new documents
new_documents = [
    "Artificial intelligence and machine learning",
    "India won the football tournament",
    "Students attend university classes"
]

new_tfidf = vectorizer.transform(new_documents)
new_similarity = cosine_similarity(new_tfidf, centroids)

new_predictions = categories[
    np.argmax(new_similarity, axis=1)
]

print("\n===== NEW DOCUMENT CLASSIFICATION =====")
for document, prediction, scores in zip(
    new_documents, new_predictions, new_similarity
):
    print("\nDocument:", document)
    print("Similarity Scores:")
    for category, score in zip(categories, scores):
        print(category, ":", round(score, 4))
    print("Predicted Category:", prediction)
```

9. Explanation of the Python program

Step 1 — Import the libraries

NumPy is used for numerical calculations and arrays. TfidfVectorizer converts text into numerical TF-IDF vectors. train_test_split divides the dataset. cosine_similarity compares vectors. accuracy_score and classification_report evaluate classification performance.

Step 2 — Create training documents and labels

The complete program contains 15 documents: five Technology, five Sports and five Education documents. Every training document has a corresponding category label. Unlike K-Means clustering, centroid-based classification requires predefined labels.

Step 3 — Divide the dataset

```
X_train, X_test, y_train, y_test = train_test_split(
    documents, labels,
    test_size=0.2,
    random_state=42,
```

```
    stratify=labels
)
```

The split uses 80% of the data for training (12 documents) and 20% for testing (3 documents). X_train contains training documents, X_test contains testing documents, y_train contains training labels, and y_test contains actual testing labels. stratify=labels preserves category proportions.

Step 4 — Convert text into TF-IDF features

```
vectorizer = TfidfVectorizer(stop_words="english")
X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)
```

The vectorizer learns the vocabulary and IDF weights from training documents. fit_transform() creates training vectors. transform() converts test documents using the same learned vocabulary and weights.

Step 5 — Identify unique categories

```
categories = np.unique(y_train)
```

This extracts the unique category names from the training labels, such as Education, Sports and Technology.

Step 6 — Calculate the centroid for each category

```
for category in categories:
    category_documents = X_train_tfidf[
        np.array(y_train) == category
    ]
    centroid = category_documents.mean(axis=0)
    centroids.append(np.asarray(centroid).ravel())
```

First, the code selects only training documents belonging to the current category. Next, mean(axis=0) calculates the average TF-IDF value for every vocabulary feature. The resulting vector is the category centroid. The loop repeats for all categories, and the centroids are combined into a matrix.

Step 7 — Calculate cosine similarity

```
similarity_matrix = cosine_similarity(
    X_test_tfidf, centroids
)
```

This calculates the similarity between every testing document and each category centroid. A higher cosine similarity indicates a closer match in direction between the two vectors.

Step 8 — Predict the category

```
predicted_indices = np.argmax(similarity_matrix, axis=1)
y_pred = categories[predicted_indices]
```

np.argmax() finds the position of the highest similarity score for each test document. The category at that position becomes the predicted category.

Step 9 — Evaluate classification performance

```
accuracy = accuracy_score(y_test, y_pred)
```

Accuracy compares predicted labels with actual test labels. classification_report() displays precision, recall, F1-score and support for each category.

Step 10 — Classify new documents

```
new_tfidf = vectorizer.transform(new_documents)
new_similarity = cosine_similarity(new_tfidf, centroids)
new_predictions = categories[
    np.argmax(new_similarity, axis=1)
]
```

New documents are converted using the training vectorizer, compared with the existing centroids and assigned to the category with the highest similarity. The model does not recalculate the centroids when predicting these new documents.

10. Expected output

The program displays the category centroids, test document predictions, classification accuracy, classification report, similarity scores and predictions for new documents. Exact numerical values depend on the generated TF-IDF features

and the training/test split.

```
===== CATEGORY CENTROIDS =====
Category: Education
Centroid: [numerical values]

Category: Sports
Centroid: [numerical values]

Category: Technology
Centroid: [numerical values]

===== TEST DOCUMENT CLASSIFICATION =====
Document: [testing document]
Actual Category: [actual label]
Predicted Category: [predicted label]

===== CLASSIFICATION PERFORMANCE =====
Accuracy: [calculated value]

===== NEW DOCUMENT CLASSIFICATION =====
Document: Artificial intelligence and machine learning
Similarity Scores:
Education : [value]
Sports : [value]
Technology : [value]
Predicted Category: Technology
```

11. How to execute the program

1. Open IDLE, VS Code, PyCharm or another Python environment.
2. Install the libraries using `pip install scikit-learn numpy`.
3. Create a Python file named **centroid_text_classification.py**.
4. Paste the complete program and save it.
5. Open a terminal in the folder containing the file and run:

```
python centroid_text_classification.py
```

> No external dataset is required because the sample documents are included in the script.

12. Difference between K-Means and Centroid-Based Classification

K-Means	Centroid-Based Classification
Unsupervised learning.	Supervised learning.
Does not require predefined category labels.	Requires predefined category labels.
Creates clusters automatically.	Calculates centroids from known categories.
Assigns documents to clusters.	Assigns documents to known categories.
Centroids are updated iteratively.	Centroids are calculated from labelled training documents.

13. Applications

• News article classification
• Research paper categorization
• Email classification
• Customer review categorization
• Document management
• Academic subject classification
• Medical document classification

14. Student exercises

1. Add a fourth category called Healthcare.
2. Add five new documents to each category.
3. Calculate centroids using Bag-of-Words instead of TF-IDF.
4. Compare cosine similarity with Euclidean distance.

5. Classify five new documents.
6. Compare centroid-based classification with SVM and Naive Bayes.

15. Quick knowledge check

Question 1: What does a centroid represent?
Answer: The average feature vector of a category.

Question 2: Which similarity measure is used in this experiment?
Answer: Cosine similarity.

Question 3: Does centroid-based classification require predefined labels?
Answer: Yes. It is supervised learning.

Expected laboratory outcome: Students should be able to convert text into TF-IDF vectors, calculate category centroids, measure cosine similarity and classify new documents based on similarity to category centroids.

Teaching suggestion: Demonstrate centroid calculation first using three simple numerical vectors. Then explain how the same averaging process is applied to TF-IDF vectors. Finally, execute the Python program and demonstrate how a new document is assigned to the most similar category.

Note: This is a small educational dataset. For reliable classification performance, use a larger and more diverse collection of labelled documents.